Industr.IA

E-book

Edge AI: continuum cloud-edge, compressão de modelos e runtimes

Unidade M4 | Capítulo Único

Prof.: Arthur Nascimento



SUMÁRIO

1. O continuum cloud-edge 3
2. Hardware e runtimes 6
3. Técnicas de compressão 10
4. Um exemplo industrial completo 15
5. Conclusão 17

Unidade M4

1. O continuum cloud-edge

Esta seção tem por objetivo que você reconheça os pontos do continuum cloud-edge e saiba situar onde a inferência de uma solução pode ocorrer. A pergunta que organiza toda a lente do deployment é simples de enunciar e difícil de responder bem: dado um modelo já treinado, onde ele deve rodar quando entrar em produção? A resposta não é única, e o erro mais frequente é tratar a nuvem como destino automático, sem perceber que existe um espectro de opções entre o data center remoto e o próprio sensor que gera o dado.

Convém fixar primeiro o vocabulário. O termo *edge*, no sentido usado aqui, não nomeia uma tecnologia, e sim uma posição: o lugar onde a inferência acontece, próximo da fonte dos dados, em vez de num servidor central distante.

GLOSSÁRIO

Edge AI

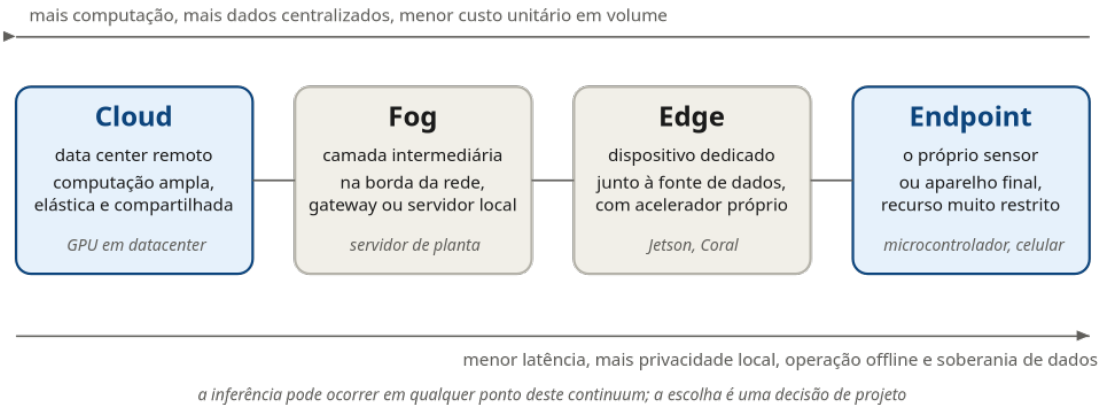
Execução de modelos de inteligência artificial em hardware próximo à fonte dos dados, como sensores, câmeras, gateways ou aparelhos do usuário, em vez de um data center remoto. Edge descreve a posição da inferência no continuum, não uma família de modelos: o mesmo modelo pode ser servido na nuvem ou no edge conforme a decisão de deployment.

Entre os dois extremos há quatro pontos de referência. Na *cloud*, a inferência ocorre num data center remoto, com computação ampla, elástica e compartilhada, mas a uma distância de rede que cobra seu preço em latência e em dependência de conectividade. Na *fog*, uma camada intermediária na borda da rede, um gateway ou servidor de planta processa os dados perto de onde foram gerados, reduzindo a viagem até a nuvem. No *edge*, um dispositivo dedicado junto à fonte, com acelerador próprio, executa o modelo localmente. No *endpoint*, o próprio sensor ou aparelho final, com recurso muito restrito, realiza a inferência sem depender de nada externo.

A fronteira entre fog e edge é mais gradual do que os nomes sugerem. Fog costuma designar um servidor ou gateway que atende vários dispositivos de uma mesma planta, enquanto edge designa o dispositivo dedicado junto a uma fonte específica. Na prática, muitas soluções combinam os dois, com sensores no endpoint, inferência no edge e

agregação na fog, e a nuvem reservada ao que não precisa de resposta imediata. A figura a seguir organiza esse espectro e os compromissos que se movem ao longo dele.

Figura 1 - Continuum cloud, fog, edge, endpoint.



Fonte: autor.

Para cego ver: Figura 1 intitulada "Continuum cloud, fog, edge, endpoint". O diagrama apresenta uma sequência horizontal composta por quatro blocos conectados, representando os diferentes níveis de processamento de dados em uma arquitetura distribuída. Na parte superior, uma seta aponta da direita para a esquerda com a indicação de que, em direção ao Cloud, há mais computação, mais dados centralizados e menor custo unitário em volume. O primeiro bloco, Cloud, descreve um data center remoto, com computação ampla, elástica e compartilhada, tendo como exemplo GPU em datacenter. Em seguida, o bloco Fog representa uma camada intermediária na borda da rede, composta por um gateway ou servidor local, exemplificado por um servidor de planta. O terceiro bloco, Edge, corresponde a um dispositivo dedicado junto à fonte de dados, equipado com acelerador próprio, tendo como exemplos Jetson e Coral. O último bloco, Endpoint, representa o próprio sensor ou aparelho final, caracterizado por possuir recursos muito restritos, como um microcontrolador ou um telefone celular. Na parte inferior, uma seta aponta da esquerda para a direita, indicando que, em direção ao Endpoint, há menor latência, maior privacidade local, operação offline e maior soberania dos dados. Abaixo da seta, uma observação informa que a inferência pode ocorrer em qualquer ponto desse continuum, sendo essa escolha uma decisão de projeto. O diagrama utiliza caixas retangulares, alinhadas horizontalmente e conectadas entre si, representando a continuidade entre os diferentes níveis da arquitetura computacional. Na parte inferior da figura consta a indicação "Fonte: autor."

O que justifica deslocar a inferência em direção ao edge não é preferência estética, e sim um conjunto de motivações concretas. A primeira é a latência: quando a resposta precisa sair em poucos milissegundos, a viagem de ida e volta até a nuvem pode ser inviável, e processar localmente elimina esse trajeto. A segunda é a privacidade: manter o dado bruto no dispositivo, sem enviá-lo a um servidor externo, reduz a superfície de exposição e simplifica o cumprimento de obrigações regulatórias. A terceira é a banda: transmitir vídeo contínuo de dezenas de câmeras para a nuvem custa caro e satura o enlace, ao passo que rodar o modelo na ponta envia apenas o resultado. A quarta é a operação offline: linhas de produção, embarcações e equipamentos em campo nem sempre têm conectividade confiável, e uma solução que só funciona conectada não funciona ali.

Próxima da privacidade, mas distinta dela, está a soberania de dados: a exigência de que certos dados permaneçam sob a jurisdição de um país ou de uma organização, por lei ou por contrato. Manter a inferência local ajuda a cumprir essa exigência, porque evita que o dado bruto cruze fronteiras para ser processado. É um fator que costuma pesar em setores regulados e em contratos com órgãos públicos, e que muitas vezes decide a favor do edge mesmo quando latência e banda não seriam impeditivas.

Vale ainda separar Edge AI de um termo próximo, TinyML. Edge AI nomeia a posição da

inferência, próxima da fonte, e vale para dispositivos de qualquer porte, de um gateway robusto a um celular. TinyML designa o caso extremo em que o modelo roda em microcontroladores de pouquíssima memória e energia, medida em miliwatts, como um sensor que reconhece uma palavra de comando sem nunca se conectar à rede. Todo TinyML é Edge AI, mas nem todo Edge AI é TinyML, e a distinção importa porque o orçamento de recursos no endpoint é muito mais apertado do que no edge.

Exemplo aplicado

Uma transportadora instala câmeras nas cabines para detectar sinais de fadiga do motorista e emitir um alerta sonoro imediato. A resposta precisa ocorrer em frações de segundo, a conectividade na estrada é intermitente e as imagens do rosto do motorista são dado sensível. Os três fatores apontam na mesma direção: a inferência roda no edge, dentro do dispositivo na cabine, e só metadados agregados sobem para a nuvem. Enviar cada quadro para um servidor remoto somaria latência incompatível com o alerta e exporia imagens que não precisam sair do veículo.

Armadilha comum

Um erro recorrente é tratar a decisão de onde a inferência ocorre como detalhe de engenharia a ser resolvido no fim, depois de escolher o modelo. A consequência é cara: um modelo dimensionado para a nuvem raramente cabe num dispositivo restrito sem retrabalho, e requisitos de latência ou privacidade descobertos tarde forçam reprojetar a solução. O posicionamento no continuum é uma decisão de produto que condiciona a escolha do modelo, e não o contrário.

Para aprofundar

Para uma introdução sistemática a inferência eficiente na ponta, veja HAN, S., TinyML and Efficient Deep Learning Computing (MIT 6.5940), materiais abertos em efficientml.ai. O tratamento aprofundado de arquiteturas de deployment em fleet e de federated learning é deferido aos cursos de Visão Computacional e Real Time Processing da Fase 2 do programa.

Fixado o continuum e as razões para percorrê-lo, falta saber em que esse modelo de fato roda na ponta.

2. Hardware e runtimes

Esta seção tem por objetivo que você reconheça, em alto nível, os runtimes de execução e os formatos de modelo apropriados a diferentes alvos de hardware, sem entrar no detalhe de implementação de cada um. A ideia central, que organiza toda a seção, é separar três coisas que costumam ser confundidas: o formato em que o modelo é distribuído, o runtime que o executa, e o hardware sobre o qual ele roda.

Convém começar pelo que um modelo é, do ponto de vista de quem vai executá-lo. Um modelo treinado não é um programa comum, e sim duas coisas combinadas. A primeira é um grafo de operações, que descreve a sequência de multiplicações de matrizes, convoluções e funções de ativação que transformam a entrada na saída. A segunda é um conjunto de pesos, os números aprendidos durante o treino que preenchem esse grafo. Treinar é ajustar os pesos até que o grafo produza boas saídas; a inferência apenas percorre o grafo uma vez, com os pesos já fixados, para responder a uma entrada nova. Guardar essa imagem ajuda a entender tudo o que vem a seguir.

Durante o treino, esse grafo vive dentro de um framework como PyTorch ou TensorFlow, amarrado às bibliotecas e à linguagem daquele ambiente. Para rodar fora dali, o modelo precisa ser exportado para um formato portável, que é um arquivo contendo o grafo e os pesos escritos de uma forma que não depende do framework de origem. Exportar é, em essência, serializar o modelo nesse arquivo. O ponto a reter é que o formato é só o arquivo: ele descreve o modelo, mas não executa nada por si.

GLOSSÁRIO

Formato de modelo

Arquivo que guarda o grafo de operações e os pesos de um modelo já treinado, de forma independente do framework em que foi criado. Formatos comuns na ponta incluem ONNX, TFLite e GGUF. O formato não roda o modelo; ele só o descreve. Quem o executa é o runtime.

O runtime é o programa que carrega esse arquivo e executa a inferência. Ao receber o grafo, ele percorre cada operação e a traduz para uma implementação otimizada para o hardware disponível, escolhendo, por exemplo, um kernel de multiplicação de matrizes que aproveite as instruções vetoriais da CPU ou os núcleos de uma GPU. Bons runtimes ainda reorganizam o grafo antes de executá-lo, fundindo operações vizinhas para reduzir acessos à memória e ganhar velocidade. Nada disso altera os pesos: o runtime não treina, apenas roda o que recebe.

GLOSSÁRIO

Runtime

Software que carrega um modelo em formato portável e executa a inferência, traduzindo as operações do modelo para o hardware disponível (CPU, GPU, NPU, TPU). Um runtime não treina o modelo; apenas o roda. Exemplos: TFLite, ONNX Runtime, Core ML, llama.cpp.

Os runtimes se dividem em dois grupos quanto à origem, e reconhecer essa divisão evita confusão na hora de ler uma especificação. Alguns são presos a um framework: o TFLite executa modelos que vieram do TensorFlow, e o Core ML, modelos preparados para o ecossistema da Apple. Outros são neutros em relação ao framework de origem, e o principal exemplo é o ONNX Runtime, que roda o formato ONNX, para o qual modelos de quase qualquer framework podem ser convertidos. A vantagem do caminho neutro é desacoplar a escolha do framework de treino da escolha do runtime de produção; a vantagem do caminho preso é a integração mais estreita com o hardware daquele ecossistema.

Na ponta, os runtimes mais comuns cobrem alvos distintos. O TFLite executa modelos no formato TensorFlow Lite em celulares e dispositivos embarcados. O ONNX Runtime roda o formato ONNX numa variedade ampla de plataformas e aceleradores. O Core ML serve os aparelhos da Apple, aproveitando o acelerador dedicado deles. Para modelos de linguagem pequenos em CPU, o llama.cpp executa o formato GGUF com eficiência notável, e o MLC LLM compila modelos para alvos diversos, incluindo navegador e celular. O quadro a seguir resume essa correspondência entre formato, runtime e alvo típico.

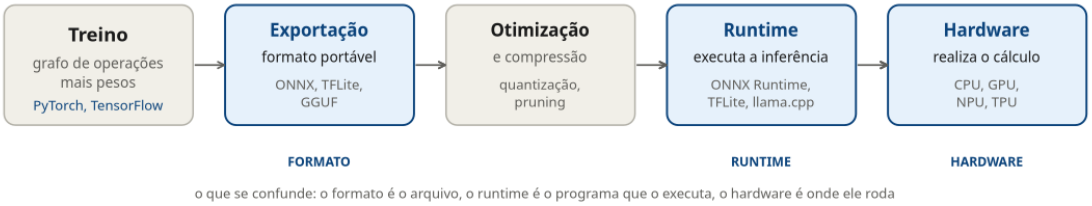
Quadro 1 - Runtimes, formatos e alvos típicos de execução na ponta.

Runtime	Formato típico	Alvo onde brilha
TFLite	TFLite (.tflite)	celular Android e microcontroladores
ONNX Runtime	ONNX (.onnx)	ampla gama de plataformas e aceleradores
Core ML	Core ML (.mlmodel)	aparelhos Apple com Neural Engine
llama.cpp	GGUF (.gguf)	LLM pequeno em CPU, sem GPU
MLC LLM	formato compilado próprio	LLM em navegador, celular e edge

Para cego ver: Quadro 1 intitulado " Runtimes, formatos e alvos típicos de execução na ponta." A figura apresenta uma tabela com três colunas: Runtime, Formato típico e Alvo onde brilha, comparando diferentes ambientes de execução utilizados para implantar modelos de inteligência artificial em dispositivos de borda (edge). Na primeira linha, o TFLite utiliza o formato TFLite (.tflite) e é indicado principalmente para celulares Android e microcontroladores. Na segunda linha, o ONNX Runtime trabalha com o formato ONNX (.onnx) e se destaca pela compatibilidade com uma ampla variedade de plataformas e aceleradores de hardware. Na terceira linha, o Core ML utiliza o formato Core ML (.mlmodel) e é otimizado para dispositivos Apple equipados com o Neural Engine. Na quarta linha, o llama.cpp utiliza o formato GGUF (.gguf) e é voltado para a execução de modelos de linguagem de pequeno porte em processadores (CPU), sem necessidade de GPU. Na quinta e última linha, o MLC LLM utiliza um formato compilado próprio e é destinado à execução de modelos de linguagem em navegadores, dispositivos móveis e ambientes de edge computing. A tabela utiliza cabeçalhos em azul-claro e organiza as informações em linhas, permitindo comparar de forma objetiva os principais runtimes, seus formatos de arquivo e os cenários em que cada um apresenta melhor desempenho.

Vale ver o caminho inteiro de uma vez, do treino ao hardware, para não confundir as etapas. A figura a seguir organiza o pipeline e destaca onde entram, respectivamente, o formato, o runtime e o hardware, que são justamente os três termos que costumam ser tratados como sinônimos.

Figura 2 - Pipeline de deployment, do treino ao hardware.



Fonte: autor.

Para cego ver: Figura 2 intitulada "Pipeline de deployment, do treino ao hardware". O diagrama apresenta um fluxo linear composto por cinco etapas conectadas por setas, representando o processo de implantação de um modelo de inteligência artificial, desde o treinamento até a execução no hardware. A primeira etapa é Treino, descrita como "grafo de operações mais pesos", com exemplos de ferramentas PyTorch e TensorFlow. Em seguida, a etapa Exportação converte o modelo para um formato portátil, utilizando formatos como ONNX, TFLite e GGUF. A terceira etapa é Otimização, que realiza compressão, quantização e pruning para reduzir o tamanho e aumentar a eficiência do modelo. Depois, a etapa Runtime é responsável por executar a inferência, utilizando ambientes como ONNX Runtime, TFLite e llama.cpp. Por fim, a etapa Hardware corresponde ao dispositivo que realiza o cálculo, podendo ser uma CPU, GPU, NPU ou TPU. Abaixo do fluxograma há uma observação explicando que o formato corresponde ao arquivo do modelo, o runtime é o programa responsável por executá-lo e o hardware é o equipamento onde a execução acontece. O diagrama utiliza caixas retangulares, organizadas horizontalmente e ligadas por setas que indicam a sequência do processo de implantação do modelo. Na parte inferior da figura consta a indicação "Fonte: autor."

Falta o último elo, o hardware que de fato executa as operações, e quatro tipos aparecem com frequência na ponta. A CPU é o processador de uso geral, sempre presente e flexível, porém a mais lenta para redes neurais, porque faz poucas operações em paralelo. A GPU tem milhares de núcleos simples e é excelente para as multiplicações de matrizes das redes, ao custo de consumo elevado. A NPU, unidade de processamento neural, é um acelerador especializado nessas mesmas multiplicações e convoluções, que troca flexibilidade por eficiência energética e entrega muitas inferências por watt. A TPU segue a mesma lógica de acelerador e, na versão de borda, costuma ser construída para aritmética inteira, o que explica por que ela brilha com modelos quantizados em INT8 e não com modelos em ponto flutuante: seus circuitos de multiplicação foram desenhados para inteiros, de modo a economizar energia e área.

GLOSSÁRIO

NPU

Unidade de processamento neural, um acelerador de hardware especializado em multiplicações de matrizes e convoluções, as operações dominantes em redes neurais. Diferente de uma CPU de uso geral, ela troca flexibilidade por eficiência energética, entregando muitas inferências por watt, o que a torna adequada a dispositivos com orçamento de energia apertado.

Do lado dos produtos, uma curadoria em alto nível ajuda a ler uma especificação sem instrumentar nenhuma plataforma, e o mais útil é enquadrar cada peça pelo tipo de acelerador que ela é. A linha Jetson, da NVIDIA, é essencialmente uma GPU embarcada, escolha frequente quando o modelo de visão é pesado e há orçamento de energia razoável. O Coral, do Google, traz uma TPU de borda orientada a INT8, com consumo baixo e bom desempenho em classificação e detecção compactas. As NPUs integradas aos sistemas em chip de celulares, e o Apple Neural Engine nos aparelhos da Apple, aceleram a inferência no próprio aparelho do usuário sem hardware adicional. A escolha entre elas depende do par tarefa-modelo e do orçamento de energia e custo, e a comparação instrumentada de cada plataforma fica para os cursos por eixo da Fase 2.

Exemplo aplicado

Uma fabricante de equipamentos agrícolas quer embarcar um detector de ervas daninhas numa pulverizadora autônoma. O modelo é uma rede convolucional de porte médio que precisa rodar a dezenas de quadros por segundo no campo, sem conexão. A combinação escolhida é um módulo Jetson com o modelo exportado para ONNX e servido pelo ONNX Runtime com aceleração de GPU embarcada, porque a tarefa de visão é pesada demais para uma TPU de borda mais modesta e a operação ocorre longe de qualquer rede.

A conversão entre formatos merece atenção porque é onde muitos projetos tropeçam. Nem toda operação de um framework tem equivalente direto no formato de destino, e um modelo que usa uma camada incomum pode falhar ao ser exportado ou exigir uma reescrita. Além disso, um formato pode ser aceito por um runtime e não por outro, e um runtime pode rodar num hardware sem aproveitar o acelerador daquele hardware. Verificar cedo a cadeia formato, runtime e hardware com o modelo real evita descobrir tarde que a combinação escolhida não fecha.

Armadilha comum

Uma confusão frequente é tratar formato e runtime como sinônimos, dizendo que um modelo roda em ONNX ou que foi convertido para llama.cpp. ONNX e GGUF são formatos; ONNX Runtime e llama.cpp são os programas que os executam. A consequência prática do equívoco é subestimar a etapa de conversão e a compatibilidade entre formato, runtime e hardware, que é justamente onde muitos projetos de deployment travam.

Para aprofundar

Para o panorama de formatos e runtimes abertos, veja a documentação do projeto ONNX Runtime e do llama.cpp, em github.com/ggerganov/llama.cpp. A otimização específica por alvo de hardware e o tuning de baixo nível são tratados nos cursos de Visão Computacional e Real Time Processing da Fase 2 do programa.

Um limite prático atravessa toda a escolha de hardware: a memória disponível no dispositivo. Um acelerador de borda tem uma fração da memória de um servidor, e o modelo, somado às ativações que ele gera durante a inferência, precisa caber ali. É esse teto que torna a compressão quase obrigatória na ponta. Muitas vezes o hardware alvo já é dado, e a pergunta deixa de ser qual acelerador usar para ser como fazer o modelo caber no que existe.

3. Técnicas de compressão

Esta seção tem por objetivo que você reconheça, em alto nível, as três famílias de compressão de modelo, quantização, pruning e knowledge distillation, entenda o propósito de cada uma e o compromisso que negocia entre acurácia, tamanho e latência. O propósito não é implementar nenhuma delas, e sim saber o que cada uma faz e antecipar o que custa quando aparece numa especificação de deployment. As três existem pela mesma razão: um modelo treinado para a nuvem quase sempre é grande demais para a ponta, e é preciso reduzi-lo sem perder o que importa.

Antes de entrar nas técnicas, convém lembrar que comprimir não é a única via para um modelo caber na ponta. Há famílias de modelos projetadas desde o início para serem pequenas e eficientes, e um modelo menor não é necessariamente pior para uma tarefa específica: para muitos problemas industriais bem delimitados, um modelo compacto bem treinado entrega, no que importa, tanto quanto um modelo grande de propósito geral. A compressão entra quando já se tem um modelo maior do que o alvo comporta; escolher de saída um modelo pequeno é uma decisão anterior, de arquitetura, e as duas abordagens se somam.

A técnica mais usada é a quantização, que reduz a precisão numérica com que os pesos do modelo são representados.

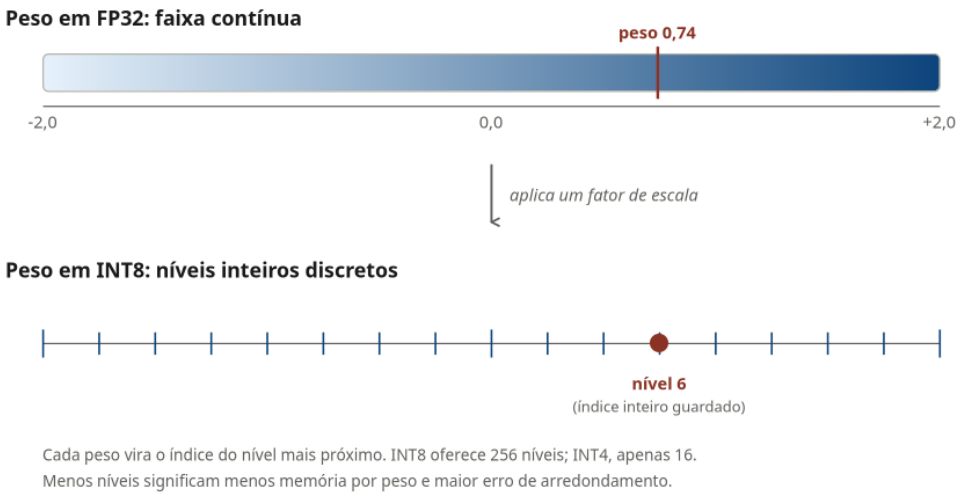
GLOSSÁRIO

Quantização

Redução da precisão numérica dos pesos e ativações de um modelo, por exemplo de ponto flutuante de 32 bits (FP32) para inteiros de 8 bits (INT8) ou 4 bits (INT4). A quantização encolhe o modelo e acelera a inferência, ao custo de uma perda de acurácia que cresce conforme a precisão diminui. FP16 costuma ser quase indolor; INT4 já exige cuidado.

Vale entender o que reduzir a precisão significa, porque é o que dá sentido ao ganho e ao custo. Um peso em ponto flutuante de 32 bits pode assumir uma faixa contínua e enorme de valores. A quantização observa a faixa real que os pesos de uma camada de fato ocupam, por exemplo de menos dois a mais dois, e a divide em um número fixo de níveis igualmente espaçados: 256 níveis no caso de INT8, apenas 16 no caso de INT4. Cada peso passa a ser guardado como o índice do nível mais próximo, e um único fator de escala, comum à camada, permite reconstruir o valor aproximado na hora de calcular. É por isso que as operações ficam mais baratas: somar e multiplicar inteiros custa menos que somar e multiplicar floats, e cada peso ocupa menos memória. A figura a seguir ilustra esse mapeamento.

Figura 3 - Intuição da quantização, faixa contínua mapeada em níveis inteiros.

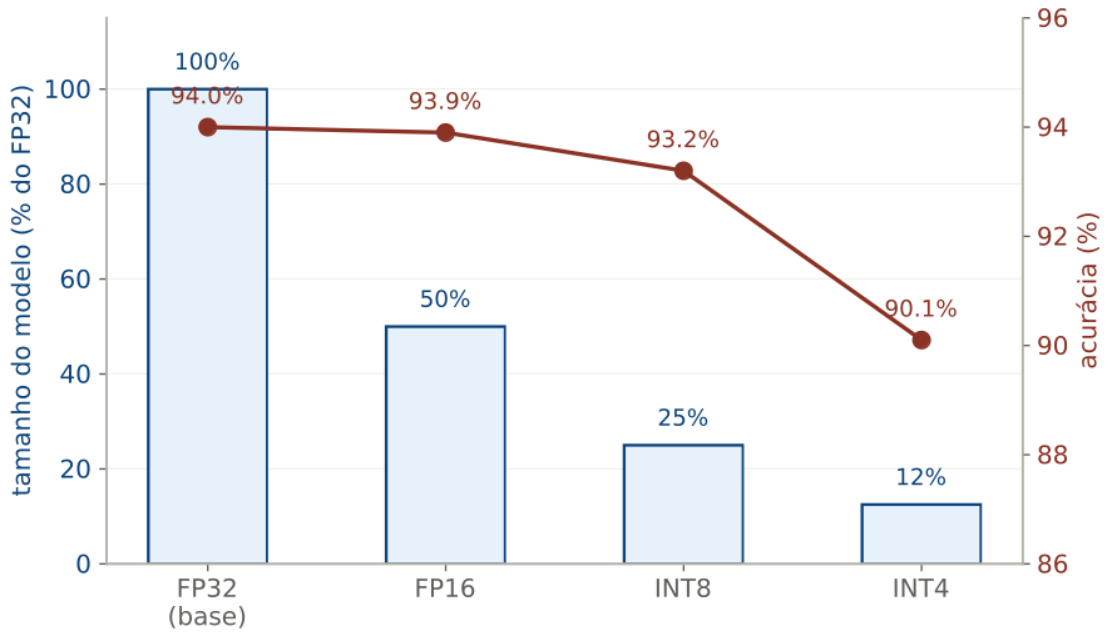


Fonte: autor.

Para cego ver: Figura 3 intitulada "Intuição da quantização, faixa contínua mapeada em níveis inteiros". O diagrama compara a representação de um peso de um modelo de inteligência artificial em ponto flutuante com sua versão quantizada em valores inteiros. Na parte superior, uma barra horizontal em representa a faixa contínua de valores em FP32, variando aproximadamente de -2,0 a +2,0. Sobre essa barra há uma linha vertical vermelha indicando o valor peso 0,74. Abaixo da barra, uma seta aponta para uma segunda representação acompanhada da indicação "aplica um fator de escala", mostrando a conversão do valor contínuo para uma representação discreta. Na parte inferior, uma linha horizontal com marcações igualmente espaçadas representa os níveis inteiros discretos em INT8. Um ponto vermelho destaca o nível 6, identificado como o índice inteiro guardado, indicando o nível inteiro mais próximo correspondente ao peso original. Abaixo do diagrama, um texto explica que cada peso é convertido para o índice do nível mais próximo, que INT8 oferece 256 níveis, enquanto INT4 oferece apenas 16, e que quanto menor o número de níveis, menor o uso de memória por peso, mas maior o erro de arredondamento. A figura utiliza barras, linhas, setas e marcadores em tons de azul e vermelho para ilustrar visualmente o processo de quantização. Na parte inferior consta a indicação "Fonte: autor.

A intuição de tamanho é direta. Um peso guardado em FP32 ocupa quatro bytes; o mesmo peso em INT8 ocupa um, e em INT4, meio. Um modelo em INT8 fica, em ordem de grandeza, com um quarto do tamanho original e roda mais rápido, porque movimenta menos memória. O preço é uma perda de acurácia que costuma ser pequena até INT8 e acelera nos regimes mais agressivos. FP16, um meio-termo em ponto flutuante de 16 bits, costuma ser quase indolor; INT4 já exige verificação cuidadosa. A figura a seguir mostra esse compromisso com dados simulados.

Figura 4 - Compromisso entre acurácia e tamanho do modelo sob diferentes regimes de quantização.



Dados simulados, ilustrativos. A queda de acurácia acelera nos regimes mais agressivos (INT4).

Fonte: autor.

Para cego ver: Figura 4 intitulada "Compromisso entre acurácia e tamanho do modelo sob diferentes regimes de quantização". O gráfico compara o tamanho relativo de um modelo de inteligência artificial e sua acurácia à medida que diferentes técnicas de quantização são aplicadas. No eixo horizontal aparecem quatro configurações: FP32 (base), FP16, INT8 e INT4. O eixo vertical à esquerda representa o tamanho do modelo, em porcentagem em relação ao modelo original em FP32, enquanto o eixo vertical à direita representa a acurácia, também em porcentagem. Barras azuis indicam o tamanho do modelo em cada configuração: FP32 corresponde a 100%, FP16 a 50%, INT8 a 25% e INT4 a 12%, evidenciando a redução progressiva do espaço ocupado pelo modelo. Sobre as barras, uma linha marrom com marcadores circulares representa a acurácia obtida em cada regime de quantização: 94,0% em FP32, 93,9% em FP16, 93,2% em INT8 e 90,1% em INT4. O gráfico mostra que a redução do tamanho do modelo ocorre de forma significativa conforme aumenta o nível de quantização, enquanto a acurácia permanece relativamente estável até INT8, apresentando uma queda mais acentuada em INT4. Na parte inferior há a observação: "Dados simulados, ilustrativos. A queda de acurácia acelera nos regimes mais agressivos (INT4)." A figura utiliza barras em azul-claro e uma linha em marrom para facilitar a comparação entre tamanho do modelo e desempenho. Na parte inferior consta a indicação "Fonte: autor."

Exemplo aplicado

Uma operadora de saneamento monitora bombas com um modelo de detecção de anomalias treinado em FP32 num servidor. Para embarcá-lo em gateways instalados nas estações, a equipe aplica quantização para INT8: o modelo encolhe a cerca de um quarto do tamanho, passa a rodar no gateway sem GPU e perde menos de um ponto de acurácia, o que é aceitável para um sistema que apenas levanta suspeitas a serem confirmadas por inspeção. Em INT4 a degradação passou a gerar alarmes falsos demais, e o regime foi descartado.

Convém saber, ainda, que a quantização pode ser feita de dois modos, apenas nomeados aqui. Na quantização pós-treino, o modelo já treinado é convertido diretamente para a precisão menor, o que é rápido mas às vezes custa acurácia. No treino consciente de

quantização, o modelo é ajustado já prevendo a perda de precisão, o que recupera parte da acurácia ao custo de um novo ciclo de treino. Qual caminho seguir é uma decisão de projeto, e o detalhe de cada um fica para os cursos por eixo da Fase 2.

Uma observação fecha o tema da quantização. Não são apenas os pesos que passam a inteiros; as ativações, os valores que circulam entre as camadas durante a inferência, também podem ser quantizadas, e é isso que permite executar a maior parte das contas em aritmética inteira. Para escolher bem os fatores de escala, costuma-se passar pelo modelo um pequeno conjunto de dados representativos e observar as faixas que os valores de fato ocupam. Esse detalhe de calibração fica para a Fase 2; aqui basta reter que a quantização toca pesos e ativações, e que a faixa observada no dado real é o que a torna confiável ou não.

As outras duas famílias atacam o tamanho por caminhos diferentes. O pruning parte da observação de que muitos pesos de uma rede treinada contribuem pouco para a saída e podem ser removidos. Há dois modos, e a diferença entre eles importa na prática. O pruning não estruturado zera pesos individuais espalhados pela rede: reduz o número de parâmetros, mas gera uma esparsidade irregular que nem todo hardware sabe aproveitar para acelerar de fato. O pruning estruturado remove unidades inteiras, neurônios, canais ou filtros, e por isso encolhe a rede de verdade e acelera a inferência em hardware comum. Em ambos os casos, remover peso costuma derrubar um pouco a acurácia, e é praxe treinar o modelo por mais algumas épocas depois da poda, o fine-tuning, para recuperar o que se perdeu.

O knowledge distillation segue uma ideia distinta: em vez de encolher um modelo grande, treina um modelo pequeno para imitá-lo. O modelo grande já treinado é o professor, e o pequeno é o aluno. O que torna a técnica eficaz é o tipo de alvo usado no treino do aluno. Em vez de aprender apenas o rótulo correto de cada exemplo, o aluno aprende com as saídas suaves do professor, isto é, a distribuição de probabilidade que ele atribui a todas as classes. Essa distribuição carrega informação mais rica do que o rótulo duro, porque revela de que outras classes cada exemplo se aproxima, e é justamente isso que permite ao aluno alcançar uma qualidade que dificilmente atingiria treinando sozinho.

GLOSSÁRIO

Knowledge distillation

Redução da precisão numérica dos pesos e ativações de um modelo, por exemplo de ponto flutuante de 32 bits (FP32) para inteiros de 8 bits (INT8) ou 4 bits (INT4). A quantização encolhe o modelo e acelera a inferência, ao custo de uma perda de acurácia que cresce conforme a precisão diminui. FP16 costuma ser quase indolor; INT4 já exige cuidado.

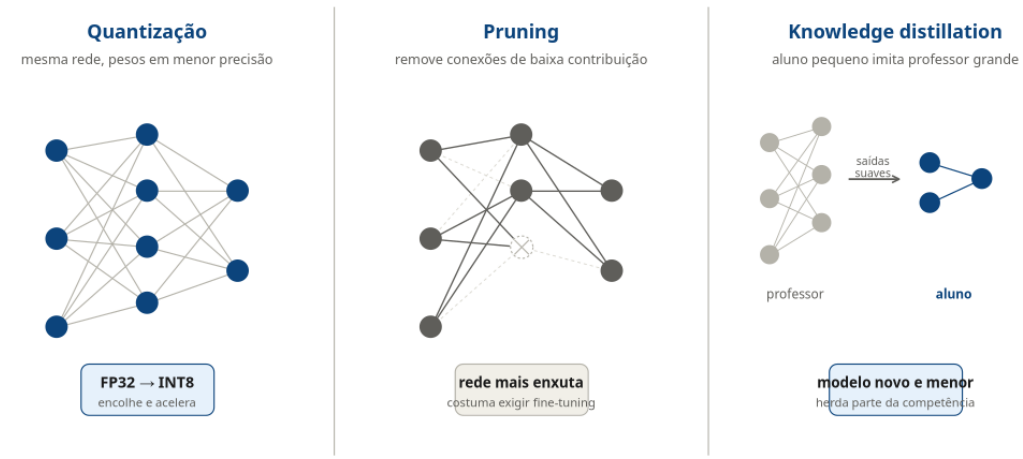
As três famílias podem inclusive ser combinadas, e na prática costumam ser encadeadas: destilar um modelo grande num aluno menor e, em seguida, quantizar o aluno para INT8, somando os dois ganhos. A escolha depende do que a aplicação tolera. O quadro a seguir reúne o que cada uma faz e o que cobra, e a figura logo depois ilustra as três lado a lado.

Quadro 2 - As três famílias de compressão, o que fazem e o que custam.

Técnica	O que faz	Custo principal	Quando escolher
Quantização	reduz a precisão numérica dos pesos	perda de acurácia nos regimes agressivos	ganho rápido de tamanho e latência
Pruning	remove pesos ou unidades de baixa contribuição	costuma exigir fine-tuning para recuperar acurácia	quando há redundância clara na rede
Knowledge distillation	treina um modelo pequeno para imitar um grande	exige treinar o aluno do zero	quando um modelo menor já basta ao alvo

Para cego ver: Quadro 2 intitulado "As três famílias de compressão, o que fazem e o que custam." A figura apresenta uma tabela com quatro colunas: Técnica, O que faz, Custo principal e Quando escolher, comparando as principais estratégias de compressão de modelos de inteligência artificial. Na primeira linha, a técnica Quantização consiste em reduzir a precisão numérica dos pesos do modelo. Seu principal custo é a possível perda de acurácia nos regimes de compressão mais agressivos. É indicada quando se busca reduzir rapidamente o tamanho do modelo e a latência de execução. Na segunda linha, a técnica Pruning remove pesos ou unidades de baixa contribuição, tornando a rede mais enxuta. Seu principal custo é que normalmente exige um processo de fine-tuning para recuperar parte da acurácia perdida. É recomendada quando existe redundância clara na estrutura da rede neural. Na terceira linha, Knowledge Distillation consiste em treinar um modelo menor para imitar o comportamento de um modelo maior, conhecido como modelo professor. Seu principal custo é a necessidade de treinar o modelo aluno desde o início. Essa técnica é indicada quando um modelo menor já é suficiente para atender aos requisitos da aplicação. A tabela utiliza cabeçalhos em azul-claro e organiza as informações em linhas, permitindo comparar de forma objetiva o funcionamento, os principais custos e os cenários de aplicação de cada técnica de compressão de modelos.

Figura 5 - As três famílias de compressão de modelo.



Fonte: autor.

Para cego ver: Figura 5 intitulada "As três famílias de compressão de modelo". O diagrama compara, lado a lado, três estratégias utilizadas para reduzir o tamanho e aumentar a eficiência de modelos de inteligência artificial: Quantização, Pruning e Knowledge Distillation. Cada estratégia é representada por uma ilustração esquemática de redes neurais. À esquerda, a seção Quantização mostra uma rede neural completa, com todos os neurônios e conexões preservados. A legenda informa "mesma rede, pesos em menor precisão". Abaixo da figura, um destaque indica "FP32 → INT8", acompanhado da explicação "encolhe e acelera", mostrando que a técnica reduz a precisão numérica dos pesos sem alterar a estrutura da rede. No centro, a seção Pruning apresenta uma rede neural semelhante, porém com algumas conexões e um neurônio representados em cinza claro e tracejado, indicando sua remoção. A legenda informa "remove conexões de baixa contribuição". Abaixo, um destaque descreve o resultado como uma "rede mais enxuta", acompanhado da observação "costuma exigir fine-tuning", indicando que o modelo normalmente precisa ser ajustado após a poda. À direita, a seção Knowledge Distillation ilustra duas redes neurais de tamanhos diferentes. À esquerda está uma rede maior, identificada como "professor", e à direita uma rede menor, identificada como "aluno". Entre elas, uma seta com a indicação "saídas suaves" representa a transferência de conhecimento do modelo maior para o menor. A legenda superior informa "aluno pequeno imita professor grande". Na parte inferior, um destaque explica que o resultado é um "modelo novo e menor", que "herda parte da competência" do modelo professor. A figura utiliza esquemas simplificados de redes neurais em tons de azul e cinza, organizados em três colunas separadas por linhas verticais, facilitando a comparação entre as técnicas de compressão de modelos. Na parte inferior consta a indicação "Fonte: autor."

Armadilha comum

A confusão decisória mais cara aqui é supor que comprimir é de graça e escolher sempre o regime mais agressivo para minimizar o tamanho. Cada nível de compressão é uma decisão entre tamanho, latência e precisão, e o equilíbrio correto depende do grau de tolerância da tarefa erro. Decidir por INT4 porque cabe melhor, sem medir a queda de acurácia no dado real da aplicação, troca um problema de espaço por um problema de qualidade que só aparece em produção.

Para aprofundar

Para os fundamentos de quantização inteira, veja JACOB, B. et al., Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, em arxiv.org/abs/1712.05877; para distillation, HINTON, G.; VINYALS, O.; DEAN, J., Distilling the Knowledge in a Neural Network, em arxiv.org/abs/1503.02531. A implementação detalhada de cada técnica é tratada nos cursos por eixo da Fase 2.

Com o continuum, os runtimes e as técnicas de compressão em mãos, é possível ler um caso real de ponta a ponta e ver como esses elementos se combinam numa decisão de deployment justificada.

4. Um exemplo industrial completo

Esta seção tem por objetivo que você leia um exemplo industrial completo de decisão Edge versus Cloud, identificando como o posicionamento no continuum, a escolha de compressão e runtime, e a estrutura de custo se articulam numa justificativa explícita. A leitura é o exercício; o preenchimento do instrumento de decisão correspondente fica para a camada decisória do módulo, tratada em objeto próprio.

O caso é o de uma linha de inspeção visual numa fábrica de autopeças. Quarenta câmeras fotografam peças usinadas em alta cadência, e um modelo de visão precisa aprovar ou reprovar cada peça em tempo de ciclo, antes que ela avance na esteira. A pergunta de deployment é onde rodar esse modelo: na nuvem, num servidor de planta, ou em dispositivos junto às câmeras.

O primeiro eixo é o posicionamento no continuum. A decisão exige resposta em poucos milissegundos por peça, e a esteira não pode esperar a viagem até um data center remoto, o que descarta a nuvem para a inferência em si. Transmitir o vídeo das quarenta câmeras para fora da planta saturaria o enlace e custaria caro em banda. Esses dois fatores empurram a inferência para o edge, em dispositivos dedicados junto às câmeras, com um servidor de planta na camada fog apenas para agregação e registro. O segundo eixo é a técnica de compressão e o runtime.

O modelo de visão é exportado para ONNX, quantizado para INT8 e servido pelo ONNX Runtime sobre uma TPU de borda Coral, combinação que cabe no orçamento de energia do dispositivo e mantém a acurácia dentro da margem tolerada pela inspeção. O terceiro eixo é a estrutura de custo total. Em regime de alto volume contínuo, a soma de inferência na nuvem mais banda superaria, ao longo do tempo, o custo fixo dos dispositivos de borda, e o caso de negócio favorece o edge. O quadro a seguir organiza essa leitura.

Quadro 3 - Leitura comparativa Edge versus Cloud para a linha de inspeção visual.

Eixo de decisão	Cloud	Edge (escolhido)
Latência por peça	incompatível com o tempo de ciclo	poucos milissegundos, local
Banda exigida	alta, vídeo de 40 câmeras	baixa, só o resultado sobe
Custo em alto volume	inferência mais banda recorrentes	custo fixo dos dispositivos
Compressão e runtime	modelo FP32 em GPU de datacenter	INT8 em Coral via ONNX Runtime

Para cego ver: Quadro 3 intitulado "Leitura comparativa Edge versus Cloud para a linha de inspeção visual." A figura apresenta uma tabela com três colunas: Eixo da decisão, Cloud e Edge (escolhido), comparando as duas abordagens para uma aplicação de inspeção visual. Na primeira linha, o eixo Latência por peça indica que, na abordagem Cloud, a latência é incompatível com o tempo de ciclo da linha de produção. Já na abordagem Edge, o processamento ocorre localmente, em poucos milissegundos, atendendo aos requisitos de tempo. Na segunda linha, o eixo Banda exigida mostra que a solução Cloud demanda alta largura de banda, exemplificada pelo envio de vídeo proveniente de 40 câmeras. Na solução Edge, a necessidade de banda é baixa, pois apenas o resultado da inferência é transmitido. Na terceira linha, o eixo Custo em alto volume destaca que, na abordagem Cloud, o custo é composto por inferência e consumo recorrente de banda, enquanto, na abordagem Edge, predomina o custo fixo dos dispositivos instalados. Na quarta e última linha, o eixo Compressão e runtime informa que a solução Cloud utiliza um modelo FP32 executado em GPU de datacenter, enquanto a solução Edge emprega um modelo quantizado em INT8 executado em um acelerador Coral por meio do ONNX Runtime. A tabela utiliza cabeçalhos em azul-claro e organiza as informações em linhas, permitindo comparar, de forma direta, os principais critérios técnicos e econômicos envolvidos na escolha entre processamento em Cloud e em Edge para aplicações de inspeção visual.

Vale registrar que o sentido da decisão não é universal. Trocar o cenário muda a resposta, e ler o caso inverso ajuda a não tomar o edge como destino automático, do mesmo modo que antes não se deveria tomar a nuvem.

Exemplo aplicado

Uma startup de laudos médicos por imagem considerou embarcar seu modelo nos aparelhos das clínicas para reduzir latência. O volume por clínica é baixo, o modelo é grande e atualizado com frequência, e a conectividade dos consultórios é boa. Aqui o edge perderia: manter e atualizar o modelo em centenas de aparelhos heterogêneos custaria mais que servi-lo na nuvem, onde uma atualização vale para todos. A inferência ficou na nuvem, e o edge foi reservado a um pré-processamento leve da imagem.

Armadilha comum

A confusão decisória central da lente do deployment é tratar a escolha de onde a inferência ocorre como otimização isolada de engenharia, e não como compromisso de produto. Quando a decisão é deixada para o time de infraestrutura no fim do projeto, requisitos de latência, privacidade e custo que deveriam ter moldado a arquitetura chegam tarde demais. O posicionamento no continuum precisa ser explicitado no escopo, junto com requisitos regulatórios e expectativas de stakeholder.

Para aprofundar

Para um catálogo de técnicas de compressão aplicadas a deployment real, veja HAN, S.; MAO, H.; DALLY, W. J., Deep Compression, em arxiv.org/abs/1510.00149. A condução completa da decisão Edge versus Cloud, com o instrumento de registro, é desenvolvida na camada decisória do módulo M4.

5. Conclusão

As quatro seções deste texto-base cobriram conjuntamente o repertório mínimo da lente do deployment: o continuum cloud-edge como espectro de posições para a inferência, com as motivações que justificam deslocá-la em direção à ponta, latência, privacidade, banda, operação offline e soberania de dados; o hardware e os runtimes que executam o modelo fora da nuvem, separando com cuidado formato, runtime e alvo; as três famílias de compressão, quantização, pruning e knowledge distillation, com o mecanismo e o compromisso que cada uma negocia; e a leitura de um exemplo industrial completo em que a decisão Edge versus Cloud foi justificada eixo a eixo. Cada bloco foi tratado em alto nível, sem instrumentar nenhuma plataforma nem implementar nenhuma técnica, com foco em entender o que cada coisa faz e em reconhecer compromissos.

O ganho prático desse vocabulário aparece no momento de delimitar a solução. Saber dizer que uma tarefa exige inferência no edge por latência e banda, servida em INT8 sobre um acelerador de borda, comunica em uma frase um conjunto de compromissos que, sem a lente, exigiria várias rodadas de esclarecimento. Exigências escondidas na escolha do local de inferência, como o orçamento de energia, a estratégia de atualização do modelo na ponta e o custo recorrente de banda, sobem à superfície no momento certo, que é o de desenhar a solução, e não o de operá-la.

A próxima peça do módulo desenvolve a camada decisória, que transforma essa leitura em escolha registrada: o instrumento de decisão Edge versus Cloud, com os três eixos vistos aqui, e a articulação da decisão de deployment ao ciclo de um projeto, em conexão direta com o encontro 6, quando a arquitetura conceitual da solução precisa fixar onde a inferência ocorre.

REFERÊNCIAS

Bibliografia mínima

JACOB, B. et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In: Conference on Computer Vision and Pattern Recognition (CVPR), 2018. Preprint em https://arxiv.org/abs/1712.05877.

HINTON, G.; VINYALS, O.; DEAN, J. Distilling the Knowledge in a Neural Network. In: NeurIPS Deep Learning Workshop, 2015. Preprint em https://arxiv.org/abs/1503.02531.

HAN, S. TinyML and Efficient Deep Learning Computing (MIT 6.5940). Materiais de curso, edição 2024. Disponível em https://efficientml.ai.

Bibliografia complementar

HAN, S.; MAO, H.; DALLY, W. J. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In: International Conference on Learning Representations (ICLR), 2016. Preprint em https://arxiv.org/abs/1510.00149.

DETTMERS, T. et al. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In: Advances in Neural Information Processing Systems (NeurIPS), 2022. Preprint em https://arxiv.org/abs/2208.07339.

FRANTAR, E. et al. GPTQ: Accurate Post-training Quantization for Generative Pre-trained Transformers. In: International Conference on Learning Representations (ICLR), 2023. Preprint em https://arxiv.org/abs/2210.17323.

GERGANOV, G. llama.cpp Project Documentation. Edição em curso. Disponível em https://github.com/ggerganov/llama.cpp.

Industr.IA

Residência em TIC 49



Executor iRede

Executor Atlântico

Coordenador Softex

Financiador MCTI Futuro

Ministério da Ciência, Tecnologia e Inovação

Governo do Brasil — Do lado do povo brasileiro

Projeto apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991.